

Data Cleaning, Clustering, and LLM Prompting — PS-SSR Pipeline Documentation

Project: PS-SSR — survey-distribution reconstruction from Chinese social media for TCM 藿香正气水 (Huoxiang Zhengqi Shui) **Source code:** `survey_ssr_exp.ipynb` (cells 1–29) + `experiments/dump_*.py`
Compiled: 2026-05-02

1. Data Cleaning

1.1 Inputs

File	What	Size
<code>data/2025-05-31-...survey_feedback_0.csv</code>	Raw survey responses (ground-truth distributions)	survey_id 1000004 — 2 sub-surveys (8.0, 9.0, 11.0); demographic + multi-choice items
<code>data/taiji_all.csv</code>	Raw social-media scrape (4 channels: Weibo, Xiaohongshu, JD, Taobao)	~few million lines (CSV is partially malformed; exact row count not parseable)

1.2 Stage 1 — Survey JSON parsing (notebook cell 1–3)

Survey rows store the questionnaire response as a JSON-encoded `content` string. The parser:

- `parse_content()` — `json.loads(content)`; on failure, fix common issues (`'` → `"`, `True/False` → `true/false`) and retry; on second failure, return empty list. Robustness: handles malformed quoting in the export.
- Filter `survey_df['survey_id'] == 1000004`.
- Walk `(survey_id, sub_survey_id, uniqueId)` tuples and accumulate each respondent's option choice into a `defaultdict(int)` per question (i.e. build the empirical answer distribution).
- Skip demographic questions (`'年龄'` in `question_text` or `'常驻'` in `question_text`).
- Keep only `q_type ∈ {single, mul}` (single-choice and multi-choice items).

Result: the 6 real survey questions in `ORIG_6` get `true_distribution = answer-count` vectors across their option sets. Sub-survey 8.0 contributes 2, 9.0 contributes 3, 11.0 contributes 1.

1.3 Stage 2 — Social-media filtering and dedup (notebook cell 5)

Pipeline applied to `taiji_all.csv`:

- Type-cast** `content_desc`, `author_name` to `str`.
- Strip hashtags** — `re.sub(r'#[?#]', '', content_desc)` (Weibo/Xiaohongshu hashtag wrappers).
- Per-channel filters** — each channel applies the same exclusion set; only Xiaohongshu and Weibo subsets are concatenated; **JD and Taobao subsets are computed but discarded** (the filter exists in the notebook but only `pd.concat([filter_df_xhs, filter_df_wb])` is kept).

Xiaohongshu filter: - `data_channel == '小红书'` - `len(content_desc) < 200` (drop long-form articles)
- `content_desc` contains `'藿香正气'` - **Exclude product-name confounders:** `'藿香茶'` (a different tea), `'廣藿香'` (a different herb name), `'藿香鱼'` (a recipe) - **Exclude medical/professional authors:** any `author_name` containing `医, 药, 疗, 健康, 养生` - **Exclude topical confounders:** `'育儿'` (parenting context), `'@'` (social mentions / re-shares)

Weibo filter: same exclusions plus `content_desc != '转发微博'` (drop bare reposts) and `len(content_desc) > 5`.

1. **Exact deduplication** — `drop_duplicates(subset=['content_desc'])`.
2. **10-character substring deduplication** — for each remaining row, extract every 10-character sliding-window substring; if any substring already appears in any prior row's substring set, drop the row. This catches near-duplicate posts that differ only in punctuation, emoji, or inserted characters (extremely common in Chinese social-media reposts and template posts).

Saved → `data/1_filter_df.csv`.

1.4 Stage 3 — Cluster-noise removal (notebook cell 19–21)

After HDBSCAN clustering (Section 2), all rows with `cluster_label == -1` (HDBSCAN's noise label) are dropped. Saved → `data/2_meaningful_df.csv`.

1.5 Counts and Yield

Stage	Rows	Channel split
Raw <code>taiji_all.csv</code> (post-CSV-parse)	unknown (malformed)	4 channels including JD/TB
After channel keep + content/author filters + dedup → <code>1_filter_df.csv</code>	27,838	Weibo 24,222 / Xiaohongshu 3,616
After HDBSCAN noise removal → <code>2_meaningful_df.csv</code>	11,873	Weibo 10,124 / Xiaohongshu 1,749
Net retention from filtered → clustered	42.7% (i.e. 57.3% of filtered posts are HDBSCAN noise)	—

The 11,873-post / 66-cluster set is the ~12K corpus used throughout the paper.

2. Clustering Algorithm

2.1 Embedding (notebook cell 12–13)

- **Model:** BAAI/bge-small-zh-v1.5 (Chinese sentence-transformer, 512-dim).
- **Batching:** `batch_size=100`, `show_progress_bar=False`.
- **Note:** the *clustering* uses `bge-small-zh-v1.5`, but the downstream SSR/anchor-similarity in `experiments/` uses the larger `bge-base-zh-v1.5` (768-dim). They are separate embedders — the small model is only for HDBSCAN; the base model is for option-anchor similarity.

2.2 Dimensionality reduction (cell 14)

- **UMAP**, parameters: `n_components=50`, `random_state=42`, `n_neighbors=15`, `min_dist=0.0`.
- Reduces 512-dim embeddings → 50-dim representation in which densities are HDBSCAN-friendly.

2.3 Clustering (cell 15)

- **HDBSCAN** (`sklearn.cluster.HDBSCAN`):
 - `min_cluster_size=50`
 - `min_samples=50`
 - `cluster_selection_epsilon=0.1`
 - `metric='euclidean'`
 - `cluster_selection_method='eom'` (excess-of-mass)
- HDBSCAN labels low-density / outlier points as `-1` ("noise"); these are dropped to form `meaningful_df`.

2.4 Results

Metric	Value
Number of clusters found	66
Noise points (dropped)	57.3% of filtered posts
Smallest cluster size	51
Largest cluster size	1,518
Median cluster size	115
Mean cluster size	179.9

Cluster sizes are heavy-tailed: a few large clusters (general Huoxiang Zhengqi Shui talk, summer-heat usage, gut-cold remedies) plus many smaller specialized clusters. The cluster-mass-weighted aggregation in M0/M1/M2 explicitly accounts for this imbalance via `cluster_weights[cid]` per question (built from per-question relevance, see notebook cell 24's *Step C/E*).

2.5 Cluster topics (cell 29)

For each of the 66 clusters, sample the first 20 comments and ask Qwen3-8B (via vLLM at 127.0.0.1:8071) to write a one-sentence Chinese topic description using `topic_only_prompt` (Section 3.1). Saved → `data/3_cluster_topics.json` (66 entries).

Sample topics (`3_cluster_topics.json`):

- Cluster 56 — "用户分享了因肠胃不适（如腹泻、呕吐、腹痛等）而使用藿香正气类产品缓解症状的经历。"
- Cluster 20 — "用户普遍将藿香正气水视为夏季必备的防暑降温饮品或药物，同时带有轻松幽默的使用场景描述。"
- Cluster 9 — "用户对含有藿香正气水味道的产品（如面膜、护肤品、内调饮品等）的使用体验和感受进行描述。"
- Cluster 29 — "用户对藿香正气水苦涩口感的普遍不满与对其药用价值的认可。"
- Cluster 11 — "用户对过期藿香正气水的使用、担忧及体验分享。"

3. Prompting Templates

All experiment prompts are in `experiments/dump_m0_m2_pmfs.py:113-130` (M1, per-cluster) and `experiments/dump_b1_c2_pmfs.py:140-151` (C2, per-post). The cluster-topic and anchor prompts are in notebook cell 26.

3.1 Cluster-topic prompt (`topic_only_prompt`) — used to populate

`3_cluster_topics.json`

你是一名用户调研分析专家。

我会给你一组来自同一个语义簇 (`cluster`) 的用户评论。

请你用一句话描述该 `cluster` 的主题 (基于整体内容, 而不是个别评论)。

请严格按以下 JSON 格式输出 (注意字段和值都必须一致) :

```
{
  "cluster_topic": "<一句话主题描述>"
}
```

下面是该 `cluster` 的评论 (每条之间用换行分隔) :

{`CLUSTER_COMMENTS`}

请严格按以下 JSON 格式输出 :

```
{
  "cluster_topic": "<一句话主题描述>"
}
```

{`CLUSTER_COMMENTS`} is the first 20 comments joined by `\n`. The reason JSON instruction is doubled (top and bottom) is to combat Qwen3's tendency to start with `<think>` chain-of-thought; the trailing instruction nudges it back to format compliance.

3.2 Anchor prompt (`anchor_prompt`) — used to materialize SSR option anchors

你是一名专业的问卷语义建模专家。

我会给你一个问卷题目和该题目的所有选项。

你的任务是为每个选项生成一个 "语义锚点句子 (`anchor sentence`)"。

要求 :

1. 每个锚点句子必须是自然语言表达, 而不是重复选项文本本身。
2. 每个锚点必须清晰表达该选项所代表的态度或立场。
3. 锚点句子要尽可能具体、生动、带情绪或明确倾向 (方便进行语义相似度计算)。
4. 输出顺序必须与选项顺序完全一致。
5. 锚点句子越像真实用户说的话越好。

请严格按以下 JSON 格式输出 :

```
{
  "anchors": [
    "<对应选项1 的 anchor sentence>",
    "<对应选项2 的 anchor sentence>",
    ...
  ]
}
```

下面是问卷题目 :

{`QUESTION`}

下面是该题目的选项 (按顺序) :

{`OPTIONS`}

Anchor sentences are then encoded with `bge-base-zh-v1.5` to produce `anchor_embeddings`, the targets that posts are SSR-matched against. Implementation: `experiments/steered_ssr.py:generate_anchors_local()`.

3.3 M1 prompt — per-cluster LLM-distribution (the "LLM simulation" used in M2 ensemble)

experiments/dump_m0_m2_pmfs.py:122-130:

```
你是一位消费者调研分析专家。

以下是某类消费者群体的社交媒体帖子样本（话题：{topic}）：
{post_text}

基于这些帖子反映的消费者特征和态度，请估计这个群体在以下问卷问题中各选项的选择比例。

问题：{question}
选项：
{options_numbered}

请严格按JSON格式输出各选项的比例（总和为1）：
{"distribution": [选项1比例, 选项2比例, ...]}
```

Slot	Source	Notes
{topic}	data/3_cluster_topics.json[cid][:60]	Truncated to 60 chars
{post_text}	n_ex=8 posts sampled from cluster (random_state=42), each truncated to 100 chars and prefixed -	One prompt sees 8 posts
{question}, {options_numbered}	The current survey question + numbered option list (1. <opt1>\n2. <opt2>...)	—
Sampling	do_sample=True, temperature=0.7, top_p=0.9, n_samples=3 generations per cluster	Final cluster PMF = mean of 3 PMFs

This is the **"LLM simulation"** prompt: one call yields one PMF for the entire cluster, conditioned on the cluster topic and 8 representative posts.

3.4 C2 prompt — per-post LLM-distribution (diagnostic ablation)

experiments/dump_b1_c2_pmfs.py:140-151:

```
根据以下社交媒体帖子内容，估计这位消费者在以下问卷问题中各选项的选择比例。

帖子：{post}

问题：{question}
选项：
{options_numbered}

请严格按JSON格式输出各选项的比例（总和为1，0到1的小数）：
{"distribution": [选项1比例, 选项2比例, ...]}

JSON：
```

Slot	Source	Notes
{post}	a single post, truncated to 200 chars	one call per post
Sampling	do_sample=True, temperature=0.7, top_p=0.9	one generation per post
Aggregation	mean of 50 per-post PMFs within each cluster, then cluster-mass weighted	n_posts=50 per cluster

C2 is **not** a paper method — it is the per-post LLM corner of a 2×2 ablation (per-cluster vs per-post × SSR vs LLM) used to justify why M2 picks per-cluster prompting (M1) over per-post (C2). With the original parser, C2 mean JS = 0.0259 (looked competitive); with a hardened multi-strategy parser (P3 rerun), C2 mean JS = **0.0328** — confirming per-cluster prompting is the correct choice for the M2 ensemble.

3.5 Parsing strategy (relevant for both M1 and C2)

The hardened parser in `experiments/dump_c2_harden.py:parse_distribution_harden()` cascades four strategies:

1. **S1** — `re.finditer` for `{...}` blocks containing "distribution", then `json.loads` and validate length.
2. **S2** — `re.finditer` for any bare `[...]` block with exactly `n_opts` numeric entries.
3. **S3** — extract the last `n_opts` numeric tokens from anywhere in the response.
4. **S4** — parse 选项1: 30%-style enumerations.

In the P3 rerun across 3000 calls: S1 hit 80.4%, S3 hit 9.0%, S2 hit 0.1%, S4 hit 0% (never needed); 10.5% remained uncoverable (genuine refusals/off-topic answers). The original parser was effectively only S2-equivalent and missed ~21% of valid responses, which fell back to a uniform PMF — accidentally regularizing C2's per-post over-confident answers and inflating its apparent JS.